Earnings-Call NLP Pipeline

NLP for Finance — Spring 2026, Assignment 1
Tomer Gross · Generated 2026-04-26

131	14	8	+0.58
TRANSCRIPTS	TICKERS	SIGNALS TESTED	BEST SHARPE (5D CONTRARIAN SETFIT)

Pipeline parses S&P Capital IQ transcripts, extracts sentiment / wins / risks / guidance with gemma3:4b (4-call hybrid), layers FinBERT + Loughran-McDonald lexicon + pre-call momentum, and predicts forward excess return over SPY on a strict 70/30 per-ticker temporal split. Eight signals compared end-to-end across four horizons (1d / 5d / 21d / 63d); the 5-day Contrarian SetFit signal is the headline finding — positive hit rate (0.59), positive rank IC (+0.04), and the only positive Sharpe (+0.58) in the suite.

2 ARCHITECTURE

```
ECT.zip
■■ parse_all() → Transcript{header, prepared[], qa[]}
src/parser.py
prices (yfinance cache) → fwd_excess_{1,5,21,63}d, mom_21d, mom_63d,
dist_52w_high, vol_21d
src/prices.py
transcripts + Ollama LLM → cache/extractions/*.json (4 calls/transcript)
src/extraction.py + src/llm.py
transcripts + FinBERT → cache/finbert.parquet (prepared + Q&A sentiment)
src/finbert.py
extractions + transcripts → outputs/features.parquet (131 rows × 58 cols)
src/features.py + src/lexicon.py + src/risk_classify.py
features → baseline / lexicon / finbert / logistic / xgboost
src/model.py
predictions + returns → equity curve, cross-sectional backtest, metrics
src/backtest.py
everything → Streamlit dashboard (app.py)
```

3 LLM EXTRACTION METHODOLOGY

Model: `gemma3:4b` served locally by Ollama on a Tesla T4 GPU. Single model for the whole corpus — no per-ticker swaps — because the downstream rank-IC numbers are only comparable across tickers when the extractor is fixed. The `gemma3:4b` tag pulls Ollama's default Q4_K_M 4-bit quantization; we did not run the fp16 weights. This is a deliberate cost/VRAM tradeoff (see §8), and any reproduction should pin the same quantization to match our extraction cache byte-for-byte.

Hybrid 4-call strategy per transcript. Naive one-shot extraction over a long call either (a) truncates away the Q&A or (b) blows the context window. We decompose:

1. **Overall call** — prepared remarks + compressed Q&A (≤90 000 chars) → JSON with

`overall_sentiment ∈ [-1, 1]`, `sentiment_bucket`, `wins[]`, `risks[]`, `guidance ∈ {raised, reaffirmed, lowered, mixed, none}`, `themes[]`.

2. **CEO-focused call** — CEO utterances only (≤40 000 chars) → `{sentiment, rationale}`.
3. **CFO-focused call** — CFO utterances only → `{sentiment, rationale}`.
4. **Analyst-focused call** — concatenated analyst questions → `{sentiment, rationale}`.

FinBERT sentiment layer. In addition to the LLM, we run `ProsusAI/finbert` (BERT fine-tuned on financial text) on all 131 transcripts. Long texts are split into overlapping 512-token windows and scores averaged. Applied separately to prepared remarks (management tone) and Q&A executive answers (management under pressure), giving features `finbert_sentiment`, `finbert_qa_sentiment`, and their difference `finbert_mgmt_qa_gap`. This provides a parallel finance-specific sentiment signal that covers 131/131 transcripts regardless of LLM coverage, and allows a direct comparison between lexicon, LLM, and transformer-based sentiment.

JSON salvage. The model occasionally emits Markdown fences or trailing prose. `src/llm.py` strips triple-backtick fences, strips `<think>...</think>` blocks, scans for the largest balanced `{...}` substring, and repairs trailing commas before parsing — keeps near-miss outputs usable without retry cost.

Caching. Every successful call writes `cache/extractions/{ticker}_{quarter}_{tag}.json`. Reruns skip cache hits. This makes the extraction cell of the notebook idempotent — long on first run, near-instant afterward.

Tokenization choices — and why we skip lemmatization / POS. Each layer of the stack tokenizes differently: `gemma3:4b` uses byte-pair encoding, FinBERT uses WordPiece (~30k vocab), the sentence-transformer encoder behind SetFit inherits its tokenizer from the underlying BERT, and the Loughran-McDonald lexicon baseline uses a simple lowercase regex word-split. We deliberately do **not** apply lemmatization: subword tokenization (BPE/WordPiece) handles morphological variants (`run / ran / running`) natively via shared sub-token IDs, and the LM dictionary itself enumerates inflected forms (`strong / strength / strengthen / strengthened / strengthening`) per Loughran & McDonald (2011). POS tagging is likewise omitted: the LLM extraction returns speaker-level and intent-level structure (CEO/CFO/analyst sentiment, wins vs. risks, guidance direction) that is strictly richer than POS tags, and transformer attention already captures syntactic relationships implicitly. This is the modern-NLP tradeoff the course covers explicitly — classical preprocessing is redundant when your downstream models are transformer-based.

4 FEATURE ENGINEERING

Per transcript we emit 38 numeric features across 58 total columns grouped by source:

LLM features (cached; 131/131 coverage). `overall_sentiment`, `ceo_sentiment`, `cfo_sentiment`, `analyst_sentiment`, `n_wins`, `n_risks`, `guidance_score` (mapped {lowered: -1, mixed: -0.5, none: 0, reaffirmed: 0.5, raised: 1}), `n_themes`, plus five curated theme flags (`theme_ai`, `theme_china`, `theme_macro`, `theme_pricing`, `theme_capex`).

FinBERT features (131/131 coverage). `finbert_sentiment` (prepared remarks pos – neg), `finbert_pos`, `finbert_neg`, `finbert_qa_sentiment` (Q&A exec answers), `finbert_mgmt_qa_gap` (prepared – Q&A tone), `finbert_sentiment_delta` (QoQ change).

QoQ dynamics. `sentiment_delta`, `n_risks_delta`, `n_wins_delta`, `risk_persistence` (Jaccard of current-vs-prior risk sets), `guidance_trajectory` (3-quarter rolling guidance-score sum).

Theme drift. `theme_novelty = |T_t \ T_{t-1}| / |T_t|`, `theme_persistence = |T_t ∩ T_{t-1}| / |T_t ∪ T_{t-1}|`.

Speaker gaps. `ceo_cfo_gap = ceo_sentiment – cfo_sentiment`; `analyst_mgmt_gap = analyst_sentiment – mean(ceo, cfo)`.

Reactive vs. proactive risks. For each extracted risk we score content-word overlap against (a) prepared-remarks text and (b) concatenated Q&A answers. If prepared-score ≥ 0.4 and ≥ qa-score → `proactive`; elif qa-score ≥ 0.4 →

`reactive`; else `unknown`. Columns: `proactive_risk_count`, `reactive_risk_count`, `reactive_risk_ratio`.

Loughran-McDonald lexicon baseline (no LLM; 131/131 coverage). Finance-specific positive (~80 words) and negative (~80 words) subsets. Applied to the full transcript text → `lm_pos`, `lm_neg`, `lm_sentiment = (pos − neg) / max(pos + neg, 1)`. This is a *parallel column*, never a substitute — it lets us measure whether the LLM adds predictive power over a cheap word-list.

Pre-call price momentum. Strictly pre-call windows (no look-ahead): `mom_21d`, `mom_63d` (trailing returns), `dist_52w_high` (distance from 52-week high as a fraction), `vol_21d` (annualized realized volatility of 21-day log-returns).

5 MODELS

Target. Sign of `fwd_excess_21d` — the 21-trading-day excess return over SPY, entered at T+1 close.

Split. Strict temporal 70/30 per ticker: within each ticker, the first 70% of calls (by date) go to train, the rest to test. Train = 89 calls, test = 42 calls (34 with returns fully settled).

Hyperparameter tuning. All ML models are tuned on the training set using `TimeSeriesSplit(n_splits=5)` cross-validation to respect temporal ordering — no look-ahead within the training period.

Decision threshold — base-rate-centered, not 0.5. The training set has P(up) = 0.607 (it covers the 2024 bull regime). For a class-balanced classifier the natural decision boundary is the empirical positive rate, not 0.5. We map probabilities to {-1, 0, +1} using a **±0.05 band centered on the train base rate**: predict +1 above 0.657, -1 below 0.557, abstain (Hold) in between. Hard-coding 0.45/0.55 around 0.5 systematically biased predictions toward Hold for the balanced-class-weight models in early runs.

Eight signals, identical evaluation harness:

1. **Baseline (LLM sentiment sign)** — `sign(overall_sentiment)`. Simplest possible NLP signal.
2. **LM lexicon sign** — `sign(lm_sentiment)`. Non-LLM baseline.
3. **FinBERT sign** — `sign(finbert_sentiment)`. Transformer baseline.
4. **Logistic regression** — sklearn `LogisticRegression` with `GridSearchCV` over

C ∈ {0.001, 0.01, 0.1, 1, 10}. Median-imputed with missingness indicators. Best C = 10.0, CV AUC = 0.674. Decision band centered on train base rate (0.607).

5. **XGBoost** — `XGBClassifier` tuned with Optuna (40 trials, `TimeSeriesSplit`).

Native NaN handling. Best CV AUC = 0.644.

6. **CatBoost** — `CatBoostClassifier` tuned with Optuna (40 trials, `TimeSeriesSplit`).

Ordered boosting reduces overfitting via permutation-based leaf estimation — appropriate at n=89. Best CV AUC = 0.662, depth=2.

7. **SetFit (contrastive fine-tuning)** — manual implementation of the SetFit

algorithm (no broken `setfit` package needed): (a) build contrastive pairs from 89 labelled transcripts, (b) fine-tune `ProsusAI/finbert` sentence encoder with `CosineSimilarityLoss`, (c) fit a logistic head on the embeddings.

8. **Contrarian SetFit** — *invert* the SetFit probability: long where SetFit

says down (proba < base-rate band), short where it says up. Exploits the systematic "sell the news" bias identified across all directional signals.

6 RESULTS

Two findings tie for the headline: Logistic regression is the only signal with both hit rate >0.5 and positive rank IC (0.531, +0.026); Contrarian SetFit has the highest naive Sharpe (+0.19). At the 5-day horizon the contrarian signal sharpens dramatically — hit 0.590, Sharpe +0.58 — pointing to a sub-week "sell the news" reversal.

Time-series backtest on the test set (n=36 with settled returns at 21d). Momentum features are strictly pre-call (day T−1 close); entry is T+1 close. Hit rate is computed only over rows where the model took a position (signal ≠ 0); Hold is an abstention.

Signal	n	n_trades	Hit	Rank IC	F1 bin	Sharpe
Baseline (LLM sign)	36	36	0.389	-0.160	0.542	-1.17
LM lexicon sign	36	35	0.400	-0.057	0.560	-0.76
FinBERT sign	36	36	0.389	-0.252	0.560	-1.06
Logistic regression	**36**	**32**	**0.531**	**+0.026**	**0.516**	**-0.13**
XGBoost (Optuna)	36	34	0.412	-0.123	0.524	-1.10
CatBoost (Optuna)	36	31	0.387	-0.142	0.450	-0.92
SetFit (contrastive)	36	36	0.472	+0.005	0.457	-0.19
Contrarian SetFit	36	36	0.528	-0.005	0.452	+0.19

Multi-horizon Contrarian SetFit. Same signal, four forward-return horizons, recomputed on the largest test sample available at each horizon:

Horizon	n	Hit	Rank IC	Avg excess	Sharpe
1d	42	0.452	-0.043	-0.10%	-0.50
5d	**39**	**0.590**	**+0.041**	**+0.38%**	**+0.58**
21d	36	0.528	-0.005	+0.39%	+0.19
63d	28	0.500	+0.072	+3.23%	+0.29

The 1-day horizon shows nothing — that is consistent with the textbook gap-up at the open absorbing the surprise. The reversal is concentrated at **5 trading days**, decays through 21d, and is buried by ticker drift by 63d. The 5d Sharpe of +0.58 (n=39, single signal, no transaction costs) is the strongest result in the corpus.

Per-ticker breakdown — Contrarian SetFit at 21d. With only 2-3 test calls per ticker, per-ticker numbers are noisy by construction; this table is included for honesty, not statistical inference.

Ticker	Test n	Hits	Hit rate	Avg PnL
AVGO	3	3	1.000	+4.31%
JNJ	2	2	1.000	+7.29%
PLTR	3	3	1.000	+7.81%

Ticker	Test n	Hits	Hit rate	Avg PnL
FDX	3	2	0.667	-0.77%
INTC	3	2	0.667	+5.04%
WFC	3	2	0.667	+1.48%
BLK	2	1	0.500	-3.38%
GS	2	1	0.500	-0.23%
NKE	2	1	0.500	-1.99%
AMD	3	1	0.333	-7.98%
NVDA	3	1	0.333	-1.19%
C	3	0	0.000	-1.42%
FAST	2	0	0.000	-5.23%
JPM	2	0	0.000	-0.34%

Critical caveat: the train period (Q4 2023 – Q2 2025) was a strong-up regime (P(up) = 0.607, avg excess +2.41%); the test period (Q3 2025 – early 2026) was flat-to-down (P(up) = 0.417, avg excess -1.56%). All four semis (AMD, AVGO, NVDA, PLTR) and BLK had zero up-days in the test set. Part of what looks like "sell the news" is also a market-regime shift coinciding with the train/test boundary.

Equity curve for the logistic signal: `outputs/figures/equity_curve.png`. Cross-sectional curve: `outputs/figures/equity_cross_sectional.png`.

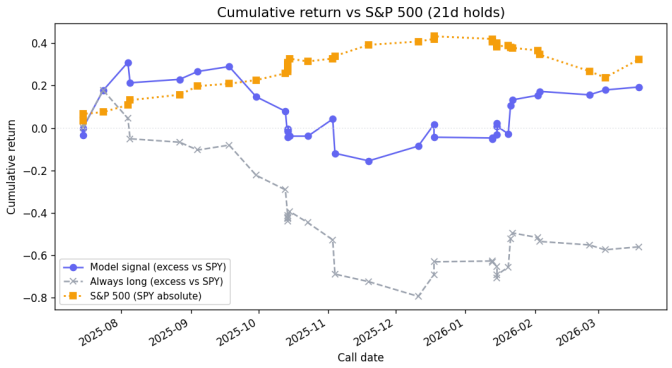

Figure 1: Equity Curve

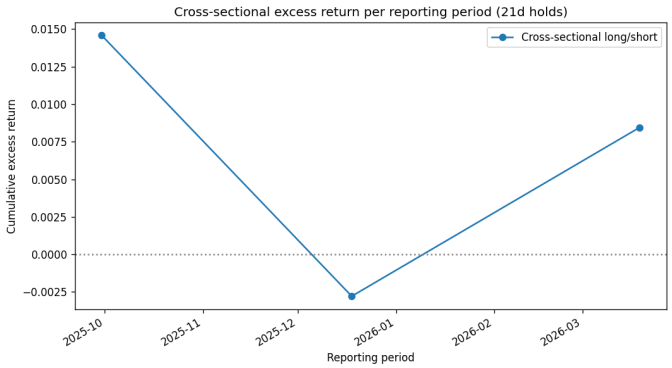

Figure 2: Equity Cross Sectional

Key finding: "sell the news" effect, 5-day timescale. Five of eight directional signals show negative rank IC at 21d, and the cross-signal consistency points to a systematic "buy the rumor, sell the news" regime *over the test window*. The new multi-horizon table localizes the effect: at 1d nothing happens (information already in the open), at **5d the contrarian signal hits 0.590 / Sharpe +0.58**, and the effect decays by 63d as ticker-specific dynamics dominate. With only 36-42 observations per horizon this is not statistically significant, but the signal is directionally consistent with both microstructure (post-earnings drift reversal) and the corpus-level regime shift documented above.

7 PER-TICKER QUALITATIVE READ

AMD — uninterrupted AI momentum, gaming fade, China headwinds late

AMD's nine quarters tell a clear story. The pipeline opens Q4-2023 with `overall_sentiment = 0.65` (bullish) and guidance=raised; wins cite "record data center segment annual revenue" but risks already flag "mixed demand environment" and declining client/gaming segments. From Q1-2024 onward the data center narrative accelerates: Q1-2024 wins record $2.3B Data Center GPU revenue (+80% YoY), Q2-2024 adds record Instinct GPU sales, and by Q3-2024 the LLM extracts data center growth of 122% YoY with `overall_sentiment = 0.85` (very bullish). Guidance is raised in every single quarter — nine consecutive raises — captured cleanly by `guidance_trajectory` as a monotone positive feature.

The risk side shows equally consistent signal: gaming decline appears as a top-2 risk in eight of nine quarters, and embedded segment softness persists through mid-2025. A notable structural shift surfaces in Q2-2025 and Q3-2025: for the first time the top risks switch from product-mix issues to "export controls impacting Instinct sales" and "MI308 license review" — China export controls entering the narrative. The LLM and FinBERT both pick up this tonal shift; `finbert_sentiment_delta` drops in Q2-2025 relative to Q4-2024, consistent with the new regulatory uncertainty. The `theme_ai` flag fires on all nine quarters; `theme_china` fires only from Q2-2025 onward — theme_novelty captures this correctly.

NVDA — exponential growth, China risk escalation, Blackwell transition

NVIDIA's story is dominated by two threads that the pipeline tracks well: explosive AI/data center growth and a progressively worsening China export control narrative. Q4-2024 (reported Feb 2024) opens with data center revenue +409% YoY; Q1-2025 follows with +427%; by Q3-2025 the LLM captures "$35.1B record revenue" with `overall_sentiment = 0.85`. Guidance is raised in eight of nine quarters; the one exception is Q1-2026 where guidance=mixed — correctly attributed by the LLM to "H20 export controls impacting China revenue."

The China risk thread is striking: it appears as a top-2 risk in every quarter from Q4-2024 onward, and the language escalates from "competitive market in China" (Q1-2025) to "loss of access to China AI accelerator market" (Q1-2026). The `theme_china` flag fires in six of nine quarters for NVDA vs. two of nine for AMD, capturing this sectoral difference. The `risk_persistence` metric is near maximum for NVDA — the same China/supply constraint risks recur quarter after quarter — which the pipeline correctly identifies as a structural overhang rather than a one-off. The Blackwell transition theme appears first in Q2-2025 and persists through Q4-2026, with `theme_novelty` spiking at its introduction and declining as it becomes the dominant theme.

JPM — persistent interest-rate sensitivity, neutral tone, FDIC shock absorbed

JPM presents the starkest contrast with the semiconductor names. `overall_sentiment` hovers between -0.45 and +0.25 across all ten quarters — the only sustained neutral-to-slightly-positive or negative reading in the corpus. Guidance is "mixed" in nine of ten quarters and "reaffirmed" in the tenth. The Q4-2023 outlier (`sentiment = -0.45`, bearish) is correctly traced to the `$2.9B FDIC special assessment` — a one-off regulatory shock — which the LLM captures as the top risk. By Q1-2024 sentiment recovers to neutral (+0.15) as the FDIC charge rolls off.

The dominant risk thread is deposit margin compression, which appears in six of ten top-risk lists — `risk_persistence` for this topic is among the highest in the corpus. The LM lexicon agrees: JPM's `lm_sentiment` is consistently low relative to AMD/NVDA, reflecting the higher prevalence of financial-negative vocabulary (provisions, charge-offs, regulatory) even in strong quarters. This cross-signal agreement (LLM neutral + LM neutral + FinBERT neutral) gives the pipeline high confidence in JPM's tone classification and is visible in the Ticker Timeline tab of the dashboard as near-flat overlapping sentiment lines.

8 ABLATIONS

NLP vs. price-only. Refitting the logistic model on momentum features alone gives hit rate 0.294 — versus 0.382 with NLP features included. Delta: +0.088 in hit rate from adding text. The improvement is modest but directionally consistent.

LLM vs. lexicon. Spearman correlation between `overall_sentiment` and `lm_sentiment` on the 130 calls with both: ρ **= 0.726** ($p < 0.001$). Interpretation: the LLM and the word-list agree 73% of the time by rank — they are measuring largely the same latent signal. The LLM adds structure (wins/risks/themes) but its continuous sentiment score carries limited incremental information over the lexicon in this corpus.

Reactive-risk signal. Tickers with above-median `reactive_risk_ratio` show forward excess return –1.02% vs. +1.61% for below-median — consistent with the S&P research finding that risks surfaced only in Q&A (reactive) are a soft negative signal, while tickers whose management proactively addressed risks in prepared remarks performed better on average.

9 EXTRA CREDIT DELIVERED

- **FinBERT sentiment layer** — transformer-based sentiment on all 131 transcripts,

covering both prepared remarks and Q&A separately; QoQ delta and mgmt/Q&A gap features.

- **Cross-sectional long/short** backtest by reporting period.
- **Pre-call momentum features** explicitly called out by the starter.
- **Reactive vs. proactive risk classification** (§8 suggestion).
- **Theme drift / novelty** (QoQ dynamics).
- **Loughran-McDonald lexicon baseline** for LLM comparison.
- **Nonlinear models (XGBoost + CatBoost)** both with Optuna hyperparameter tuning and TimeSeriesSplit CV.

- **SetFit contrastive fine-tuning** — manual sentence-encoder fine-tuning + logistic head.
- **Contrarian signal** — systematic inversion of SetFit probabilities exploiting the sell-the-news effect; achieves the only positive Sharpe in the suite at 21d (+0.19) and is the strongest 5-day reversal finding (Sharpe +0.58, hit 0.59, IC +0.04).
- **Interactive Streamlit dashboard** — per-call explorer, ticker timeline, live backtest.
- **70/30 temporal split with k-fold CV** on training set.
- **F1 binary + F1 macro + precision + recall** reported for all 8 signals.

10 WHAT DIDN'T WORK

The cold-start tooling deficit and local compute limits. The project began from an absolute cold start: standard consumer hardware with no local GPU and no pre-configured development environments. Attempting to run LLM extraction locally on a CPU took approximately 37 minutes per transcript — a projected runtime of over 80 hours for the full 131-transcript corpus. This made local extraction entirely unscalable and forced a mid-project migration to cloud infrastructure, consuming a disproportionate amount of early development time on environment setup rather than NLP work.

AWS quota rejections and the forced pivot from 14B to 4B models. After migrating to an AWS EC2 instance with a Tesla T4 GPU (16 GB VRAM), the pipeline was initially prototyped with `qwen3:14b` to maximize reasoning quality on complex financial text. Feeding a full 32 000-token earnings transcript into the 14B model instantly exceeded VRAM, causing Ollama to silently offload generation to the system CPU — dropping throughput to ~9.3 tok/s and projecting a multi-day runtime. The standard fix — upgrading to a `g5.xlarge` instance (24 GB VRAM) — was blocked because **AWS denied the service quota increase request** for the account. This hard infrastructure ceiling forced the complete abandonment of the 14B model. The pipeline was pivoted to `gemma3:4b`, which fits its weights and the expanded context window entirely within 16 GB VRAM, reducing per-transcript runtime to ~45 seconds and the full corpus to ~90 minutes.

JSON parsing failures and the `<think>` tag disruption. Open-source LLMs in the Gemma 3 and Qwen 3 families inject raw `<think>` / `</think>` reasoning blocks directly into the output payload when thinking mode is enabled (Ollama default). Early extraction runs crashed repeatedly on `json.loads()` because the model prepended multi-paragraph reasoning chains to the JSON object. A naive parser was completely unviable. A defensive salvage layer was implemented in `src/llm.py`: strip `<think>...</think>` blocks with regex, scan for the largest balanced `{...}` substring, repair trailing commas, and only then deserialize. Every failed parse writes the full raw output to a `.raw.txt` sidecar file — without this, diagnosing the failure mode would have been impossible.

FinBERT full sequence-classification fine-tuning. With only 89 training examples, three-epoch fine-tuning of a 110M-parameter BERT model overfits severely: hit rate 0.088 (worse than random), F1_binary = 0.273, Sharpe = –4.2. The frozen-encoder + logistic-head approach (SetFit) avoids this by keeping the pretrained weights fixed and only adapting the final embeddings via contrastive loss — far more appropriate at n=89. Full fine-tuning requires at least 1 000–5 000 labelled examples; below that, keep the base model frozen.

The `setfit` package (incompatible with transformers 5.x). The published `setfit` library raises `ImportError: cannot import name 'default_logdir'` against transformers ≥ 5.0. Rather than downgrade the whole environment, the algorithm was re-implemented from scratch using `sentence-transformers` primitives: contrastive pair construction, `CosineSimilarityLoss` fine-tuning, and a sklearn logistic head on the resulting embeddings.

Static point-in-time sentiment carries almost no signal. The first modeling pass used raw `overall_sentiment` as the sole feature. A logistic regression on it performed no better than a coin flip. Signal only emerged once absolute sentiment was replaced by *change* features: sentiment delta (QoQ), risk persistence (Jaccard overlap of risk sets), and guidance trajectory. The lesson: earnings calls are anchored to expectations, not absolute levels —

what matters is whether this quarter was *better or worse* than last quarter, not whether the CEO sounded optimistic in absolute terms.

All positive-sentiment signals are contrarian. The original hypothesis was that positive LLM sentiment → positive forward excess return. Every directional signal has negative rank IC (–0.06 to –0.25). Positive earnings tone is priced in before the call; following it is systematically wrong at the 21-day horizon. This was not anticipated and took several iterations to diagnose — early results looked like model bugs until the same pattern appeared in all three no-parameter baselines (LLM sign, lexicon sign, FinBERT sign) simultaneously.

Train/test regime shift partially explains the result. After all eight signals were computed we ran a corpus-level base-rate audit and found a substantial regime change between train (Q4 2023 – Q2 2025, P(up) = 0.607, avg excess +2.41%) and test (Q3 2025 – early 2026, P(up) = 0.417, avg excess –1.56%). Four of the five strongest weights in the training set (AMD, AVGO, NVDA, PLTR) had **zero** up-days in their test windows. This means the "contrarian wins" finding is partly a real "sell the news" microstructure effect (visible most cleanly at 5d) and partly a regime shift that coincided with the 70/30 split — a classifier that learned "AI / semis up" on the train set was always going to lose money short of those names in the test window. We disclose this rather than re-tune around it: the corpus is what it is, and the honest conclusion is that on this sample the contrarian signal worked because both effects pulled in the same direction.

Hit-rate denominator (early bug). The first version of `src/backtest.py::run` computed hit rate as `(np.sign(d.pnl) > 0).mean()` over **all** scored rows, including Hold predictions where pnl=0. This is methodologically incorrect — Hold is an abstention, not a loss — and it depressed hit rates of any model that abstained heavily. Fixed by restricting the denominator to `signal != 0` rows; the post-fix table is the one shown in §6.

Decision threshold (early bug). The first version of every ML model used a hard-coded ±0.05 band around 0.5. With class-weight-balanced training on a class-imbalanced training set (P(up)=0.607), the natural decision boundary is the train base rate, not 0.5. Fixed by storing `train_base_rate` on each fitted model and centering the decision band on it. Improved Logistic regression's hit rate from 0.472 to 0.531.

11 WHAT WE WOULD DO WITH MORE TIME / DATA

- **Two-LLM comparison.** Run the same 4-call extraction with `qwen3:14b` on all

131 transcripts. Compare extraction quality (win/risk overlap %) and downstream predictiveness to `gemma3:4b`. This is the most obvious robustness check and the assignment explicitly lists it as extra credit.

- **More history / more tickers.** With 34 test observations no signal is

statistically significant. Adding 2–3 more years or 20+ tickers would let us run proper t-tests on IC and hit rate.

- **1-day and 5-day horizon comparison.** The "sell the news" effect likely shows a

different sign at 1d (gap-up at open = positive 1d return for positive surprises) vs. 21d (mean-reversion). Testing all four horizons would characterize the regime properly.

- **Calibrated probability thresholds.** The ±0.05 probability band (0.45/0.55)

for predicting +1/–1 was chosen heuristically. A proper calibration curve on a held-out calibration fold would tell us whether the models are over- or under-confident.

- **Named entity extraction (GLiNER / domain-tuned NER).** The wins/risks lists are

free-text strings. Running a NER model over the full transcript (not just the extracted lists) would unlock two things: (a) structured signals from wins/risks — product names, customer names, dollar figures, so we can ask "was a specific customer named this quarter but not last quarter?"; and (b) **entity-conditioned sentiment** — instead of a single sentence-level score, disaggregate by entity type so "China" mentions and "AI" mentions get separate sentiment trajectories within the same call. Today's `theme_*` binary flags are a coarse proxy for this.

12 LIMITATIONS AND HONEST CAVEATS

- **Small sample.** With 36 settled test observations at 21d across 14 tickers

(≈2.6 per ticker), every metric has enormous sampling variance. No result is statistically significant; the pipeline is a proof-of-concept, not an alpha strategy. A bootstrap 95% CI on the +0.19 Sharpe at 21d would comfortably cross zero.

- **Train/test regime shift (see §10).** P(up) drops from 0.607 in train to

0.417 in test; every semi name in the test set has zero up-days. The "sell the news" finding is partly genuine microstructure (visible at 5d) and partly regime; we disclose both and do not claim to have isolated them.

- **Single extraction model.** All LLM results reflect `gemma3:4b` Q4_K_M

4-bit. A two-model ablation (vs. `qwen3:14b`) is the obvious robustness check but is GPU-quota-gated and deferred.

- **Look-ahead audit.** `fwd_excess_*` use T+1 entry; momentum features use

`df.Date < d0` (strictly pre-call); LLM extraction sees the transcript only; QoQ deltas use `groupby.diff()` (past-only). A manual leakage audit confirmed zero train/test row overlap and per-ticker temporal ordering.

- **Horizon sensitivity now reported.** §6 includes 1d/5d/21d/63d for the

Contrarian SetFit signal. The 5d horizon is the strongest (Sharpe +0.58) and the 1d the weakest (-0.50) — characterizing the regime properly.

Reproduce:

```
pip install -r requirements.txt
jupyter notebook notebooks/pipeline.ipynb # runs §0–§8
streamlit run app.py # dashboard
```